

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Кафедра Вычислительной техники

ОТЧЁТ ПО РАСЧЕТНО-ГРАФИЧЕСКОЙ РАБОТЕ
ПО ДИСЦИПЛИНЕ «ИНФОРМАТИКА»
«Использование графической библиотеки GLFW»

Факультет: АВТ Группа: ДТ-460а Студент: Пантюхин А.Е.	Преподаватель: Копылова О.А.
---	------------------------------

Содержание

<u>Задание.....</u>	<u>3</u>
<u>Идеи и методы решений.....</u>	<u>3</u>
<u>Функциональное описание программы</u>	<u>4</u>
<u>Основные классы и их компоненты:.....</u>	<u>4</u>
<u>Функции рисования.....</u>	<u>9</u>
<u>Движение автомобиля.....</u>	<u>11</u>
<u>Пример работы программы.....</u>	<u>12</u>
<u>Ошибки и неточности</u>	<u>13</u>
<u>Вывод.....</u>	<u>13</u>
<u>Список литературы.....</u>	<u>14</u>
<u>Приложение А. Исходный код программы.....</u>	<u>15</u>

Задание

Вариант 17. Движущийся по дороге автомобиль.

Идеи и методы решений

Было принято решение реализовать задание в трёхмерном пространстве.

В качестве основной библиотеки для работы с графикой используется GLFW, также при помощи `git submodule` и настроек CMake в проект добавлены следующие библиотеки:

`glm`: библиотека, содержащая набор функций, операторов и структур данных для решения задач векторной алгебры, стоящих в рамках реализации проекта. Кроме того, структуры `glm` полностью идентичны таковым у GLFW с точки зрения организации в памяти, за счёт чего упрощается задача по буферизации данных перед отрисовкой.

`glad`: заголовочный файл, дающий доступ к адресному пространству GL, а так же обеспечивающий кроссплатформенность нашего приложения.

`Assimp`: библиотека, реализующая функции импорта моделей в программу.

`Stb_image.h`: заголовочный файл, разворачивающийся в `.cpp` на этапе компиляции. Реализует функционал импорта текстур в программу.

Логика работы

Перед нами стоит три основные задачи:

1) Обеспечить отрисовку импортируемых моделей в трёхмерном пространстве.

2) Реализовать функционал класса «автомобиль» с возможностью управления с клавиатуры

3) Реализовать функционал класса камеры, следующей за автомобилем.

Функциональное описание программы

Основные классы и их компоненты:

Класс Mesh:

Хранит `std::vector` – переменные с информацией о точках, образующих модель, а так же текстурах, использующихся в её материале.

`std::vector<Vertex> vertices` – вектор структур, описанных ниже, состоящих их положения точки в пространстве, нормали, соответствующей ей на модели, координат этой точки в пространстве текстур и цвета точки, если текстура не используется.

`std::vector<uint32_t> indices` – вектор переменных, хранящих положительное целое число. Данные числа – индексы вертексов из массива выше, использующиеся для определения порядка их использования и предотвращения появления дубликатов вертексов с точки зрения их позиции и других свойств.

`std::vector<Texture> textures` – вектор текстур, несмотря на то, что сами текстуры там не хранятся. Данный вектор содержит в себе структуру `Texture`, а уже она определяет имя структуры в пространстве памяти GLFW, её тип, ширину, высоту и количество каналов (R, G, RG, RGB, RGBA). `std::string path` в ней используется при импорте модели для предотвращения множественной загрузки одной и той же текстуры для разных моделей. В случае возникновения такой ситуации, модели будут иметь одну и ту же текстуру, в структуре будет указан один и тот же ID.

```
namespace Engine::Render {
    enum TextureTypes {
        TEXTURE_DIFFUSE_T = 1 << 0,
        TEXTURE_SPECULAR_T = 1 << 1,
        TEXTURE_NORMAL_T = 1 << 2,
        TEXTURE_HEIGHT_T = 1 << 3,
    };

    struct Vertex {
        glm::vec3 Pos;
        glm::vec3 Normal;
        glm::vec2 UV;
        glm::vec4 Color;
    };

    struct Texture {
        uint32_t id;
    };
}
```

```

        TextureTypes type;
        int width, height, channels;
        std::string path;
    };

    class Mesh {
    public:
        std::vector<Vertex> vertices;
        std::vector<uint32_t> indices;
        std::vector<Texture> textures;

        Mesh(const std::vector<Vertex> &vertices, const
std::vector<uint32_t> &indices, const std::vector<Texture> &textures);
        void Draw(Shader &shader);
        uint32_t VAO;

    private:
        uint32_t VBO, EBO;
        void construct();
    };
}

```

Класс Model:

Данный класс является наиболее часто используемым в проекте. Он занимается импортом моделей из файлов (вызывает функции assimp и stbi для подобных задач), а так же описывает внутреннюю структуру отдельных сеток в составе модели, храня их список. При вызове метода `Model::Draw(Shader& shader)` передаёт отрисовку далее каждой из сеток в векторе `std::vector<Mesh> meshes`.

```

namespace Engine::Render {
    class Model {
    public:
        Model(const char* path);
        void Draw(Shader &shader);
        std::vector<Mesh> meshes;

    private:
        std::vector<Texture> loadedTextures;
        std::string dir;

        void load(const char* path);
        void processNode(aiNode* node, const aiScene* scene);
        Mesh processMesh(aiMesh* mesh, const aiScene* scene);
        std::vector<Texture> loadMaterialTextures(aiMaterial* mat,
aiTextureType aiType, TextureTypes engineType);
    };

    uint32_t LoadTexture(const char* path, const std::string& dir);
}

```

Класс Shader:

В данном классе реализуется логика загрузки шейдеров из файла при помощи `std::stringstream`, инициализация шейдеров, сборка программы GLSL

в комплекте из vertex и fragment шейдеров, базовые функции для передачи uniform переменных в скомпилированную программу GLSL, а так же хранится имя шейдера в адресном пространстве GLFW.

```
namespace Engine::Render {
    class Shader {
    public:
        uint32_t ID;
        Shader(const char* vertexShaderPath, const char*
fragmentShaderPath);
        void use();
        void setBool(const std::string &name, bool value) const;
        void setInt(const std::string &name, int value) const;
        void setFloat(const std::string &name, float value) const;
        void setVec2(const std::string &name, const glm::vec2
&value) const;
        void setVec3(const std::string &name, const glm::vec3
&value) const;
        void setVec4(const std::string &name, const glm::vec4
&value) const;
        void setMat2(const std::string &name, const glm::mat2
&value) const;
        void setMat3(const std::string &name, const glm::mat3
&value) const;
        void setMat4(const std::string &name, const glm::mat4
&value) const;

    private:
        enum ShaderType {
            SHADER_T_VERT = 1 << 0,
            SHADER_T_FRAG = 1 << 1,
        };

        std::string readShader(const char* path, ShaderType type);
    };
}
```

Классы сцены связаны следующим образом:

GameObject -> (CarCamera, Car), где GameObject хранит экземпляр Transform

GameObject:

Определяет виртуальные функции initialize() и run() для всех наследников, имеет базовые конструкторы для создания Transform.

```
namespace JustDrive {
    class GameObject {
    public:
        GameObject();
        GameObject(glm::vec3 position);
        GameObject(Components::Transform transform);
        Components::Transform transform;
        virtual void initialize(GLFWwindow* window) {};
        virtual void run(GLFWwindow* window, float deltaTime) {};
    };
}
```

```
}
```

Transform:

Данный класс служит, главным образом, для построения матрицы модели в ходе передачи данных из основного цикла отрисовки в шейдер-программу.

Хранит в себе векторы позиции, поворота и изменения размеры, а так же векторы ориентации forward, up, right.

Данные инкапсулированы, поскольку прямое изменение forward, up, right не предполагается, а scale, rotation – потребуют пересчёта векторов ориентации при изменении. В отсутствие в языке функционала get(); set() такое решение было сочтено удовлетворительным.

```
namespace JustDrive::Components {
    class Transform {
    public:
        Transform();
        Transform(glm::vec3 position);

        glm::vec3 getPosition();
        void setPosition(glm::vec3 position);
        glm::vec3 getRotation();
        void setRotation(glm::vec3 radians);
        glm::vec3 getScale();
        void setScale(glm::vec3 axis_scale);

        glm::vec3 getForward();
        glm::vec3 getUp();
        glm::vec3 getRight();
        glm::mat4 getTransform();

    private:
        glm::vec3 position;
        glm::vec3 rotation;
        glm::vec3 scale;

        glm::vec3 forward;
        glm::vec3 up;
        glm::vec3 right;

        void recountVectors();

    };
}
```

Класс Car:

Данный класс реализует обработку пользовательского ввода для управления автомобилем и расчёты координат и поворота автомобиля в методе run(), за счёт наследования от GameObject хранит экземпляр Transform. Также хранятся переменные, участвующие в расчётах движения автомобиля.

```

namespace JustDrive {
    class Car : public GameObject {
    public:
        Car(
            glm::vec3 position,
            glm::vec3 rotation_rad,
            float maxSpeedForward,
            float maxSpeedbackward,
            float acceleration,
            float tractionParallel,
            float tractionCross,
            float maxWheelAngle
        );
        virtual void initialize(GLFWwindow* window) override;
        virtual void run(GLFWwindow* window, float deltaTime)
override;
        glm::mat4 getTransform();
    private:
        float maxSpeedForward;
        float maxSpeedBackward;
        float acceleration;
        float tractionParallel;
        float tractionCross;
        float maxWheelAngle;
        glm::vec3 currentVelocity;
        float currentWheelAngle;
    };
}

```

Класс CarCamera:

Данный класс, аналогично классу Car, имеет свой собственный Transform, переменные и методы, необходимые в ходе отрисовки (yFov, getViewMatrix). Также в момент инициализации получает переменные lerpMultiplier и minDistance, где первая отвечает за скорость интерполяции позиции камеры от текущей до целевой, а вторая – за минимальную дистанцию до трансформы объекта слежения, поскольку целевая позиция определяется исходя из взаиморасположения объектов.

```

namespace JustDrive {
    class CarCamera : public GameObject {
    public:
        CarCamera(glm::vec3 position, float lerpMultiplier, float
minDistance, Components::Transform* targetPosition);
        virtual void initialize(GLFWwindow* window) override;
        virtual void run(GLFWwindow* window, float deltaTime)
override;
        glm::mat4 getViewMatrix();
        float yFov;
    private:
        float lerpMultiplier;
        float minDistance;
        Components::Transform* targetPosition;
    };
}

```

```
}  
};  
}
```

Функции рисования

Каждая модель проходит несколько стадий импорта и отрисовки:

1) В ходе импорта в проект, метод `void Model::load(const char* path)` загружает в память модель как сцену внутреннего формата библиотеки `assimp`. Данный метод вызывается в конструкторе класса `Model`.

2) После загрузки `assimp`-сцены, каждый «узел» в этой сцене рекурсивно обрабатывается методом `void Model::processNode(aiNode*, const aiScene* scene)`,

которая вызывает обработку каждого экземпляра сетки в данном узле и вызывает `processNode()` для всех дочерних узлов.

3) Обработка экземпляра сетки реализована методом `Mesh Model::processMesh(aiMesh* mesh, const aiScene* scene)`. Данный метод извлекает из структуры `aiMesh*` информацию о точках данной сетке, полигонах, которые они образуют, а так же текстурах, использующихся на данных точках.

Информация о точке хранится в нашей структуре `Vertex`, состоящей из `std::vector`, где хранятся позиции, нормали, UV-координаты и цвета точки.

3.5) При наличии текстур у данной сетки, также будет вызвана функция `LoadMaterialTextures(aiMaterial* mat, aiTextureType aiType, TextureTypes engineType)`, которая загрузит текстуры в память при помощи `stbi_load()`, сохранит данные и очистит внутренний буфер `stbi`.

4) При инициализации класса `Mesh` данные точек будут скопированы из пространства функции `processMesh()`, далее в методе `Mesh::construct()` (также вызывается в конструкторе) будет составлено представление данных GLFW в виде буфера `VAO(VertexArrayObject)`, состоящего из `VBO (VertexBufferObject)` с информацией о точках, а так же `EBO (ElementBufferObject)` с информацией о порядке обхода данных точек в процессе отрисовки.

5) Возвращённый экземпляр класса Mesh сохраняется в `std::vector<Mesh>` нашей модели, будучи подготовленным к отрисовке методом `Mesh::Draw(Shader& shader)` в любой момент.

Функции инициализации и управления

Основной функцией инициализации для окна GLFW является функция `GLFWwindow* GLFW::bootstrap(const char* title)`. В данной функции создаётся окно программы, устанавливается контекст GLFW, назначаются коллбеки для изменения размеров окна и выхода из программы. Далее данная функция выполняет загрузку `glad` и адресного пространства GLFW для дальнейшей работы, устанавливает параметр вертикальной синхронизации и логику работы импортера `stbi`. Если в ходе выполнения не произошло ошибок, функция отдаст указатель на экземпляр `GLFWwindow`.

`int main(void)` – Основное тело программы, вызывающее `GLFW::bootstrap()`. В нём также вызывается загрузка шейдеров и моделей для использования в цикле отрисовки (далее – в сцене), инициализируются экземпляры классов, использующихся на сцене.

Внутри основного тела существует цикл

```
while (!glfwWindowShouldClose(window))
```

Данный цикл занимается покадровой отрисовкой сцены, передаёт в экземпляры классов сцены параметры времени кадра и окна (для работы со вводом), а так же вызывает методы `Draw()` у используемых моделей. Перед вызовом метода `Draw()` шейдерам назначаются параметры `mModel`, `mView` и `mProjection` – матрицы 4x4, описывающие пространственные характеристики объекта (каждая – размер, поворот, позиция, искривление). Данные матрицы умножаются в порядке `mProjection*mModel*mView`, обеспечивающим последовательное и корректное преобразование из пространства координат сцены в пространство координат GLFW [1.0;1.0] по всем осям, и далее – в пространство нашего экрана, говоря проще – «плоскую» картинку.

Управление ходом работы программы осуществляется двумя способами:

Ход работы основного цикла отрисовки (сцены) завязан на коллбек, назначенный при инициализации. При нажатии `Escape` пользователь

посылает сигнал к остановке работы программы, и по завершении обработки кадра программа завершит свою работу.

Управление же автомобилем реализуется методом `Car::run(GLFWwindow* window, float deltaTime)`, унаследованным и перезаписанным от класса-прародителя `GameObject`, обеспечивающего наличие у потомков функций `initialize()`, `run()` и экземпляра класса `Components::Transform` для работы с пространственными характеристиками объектов сцены.

Внутри метода `Car::run()` выполняется вычисление вектора ввода пользователя – в зависимости от того, нажата ли кнопка в текущем кадре, определяется состояние нормализованного вектора ввода. Далее данный вектор используется при расчетах ускорения и замедления автомобиля в текущем кадре, а так же в расчете поворота корпуса автомобиля за кадр.

Движение автомобиля

Логика работы с пользовательским вводом уже была описана выше, приведём описание логики работы с компонентом `Components::Transform` нашего автомобиля, тем, что отвечает за его движение и поворот: Как описано выше, класс `Car` имеет несколько параметров. В начале, мы используем параметр `acceleration` для вычисления ускорения за данный кадр. Автомобиль ускорится вперёд только тогда, когда *x*-компонента вектора ввода равна единице, а текущая скорость меньше максимально доступной вперёд. Назад работает с аналогичной логикой, но уже в зависимости от максимальной скорости назад и отрицательных значений *x*-компоненты вектора ввода.

После этого, `glm::vec3 currentVelocity` – наша скорость в локальных координатах, суммируется с вычисленным ускорением по оси *Z*. В силу особенностей импортированной модели автомобиля, вектор скорости приходится умножить на вектор `transform.getRight()` вместо `transform.getForward()`, а после – инвертировать только его *x*-компоненту (вектор «вправо» модели при импорте вычислялся так, будто бы она перевернута, и её ось *Y* имеет положительное направление вниз).

Поворот автомобиля осуществляется `transform.setRotation(glm::vec3)`, куда мы направляем в качестве `rvalue` только что сконструированный вектор $0, < \text{угол поворота колёс} * \text{currentVelocity.z} * 0.5 >, 0$ в радианах. Цикл повторяется на следующем кадре.

Пример работы программы

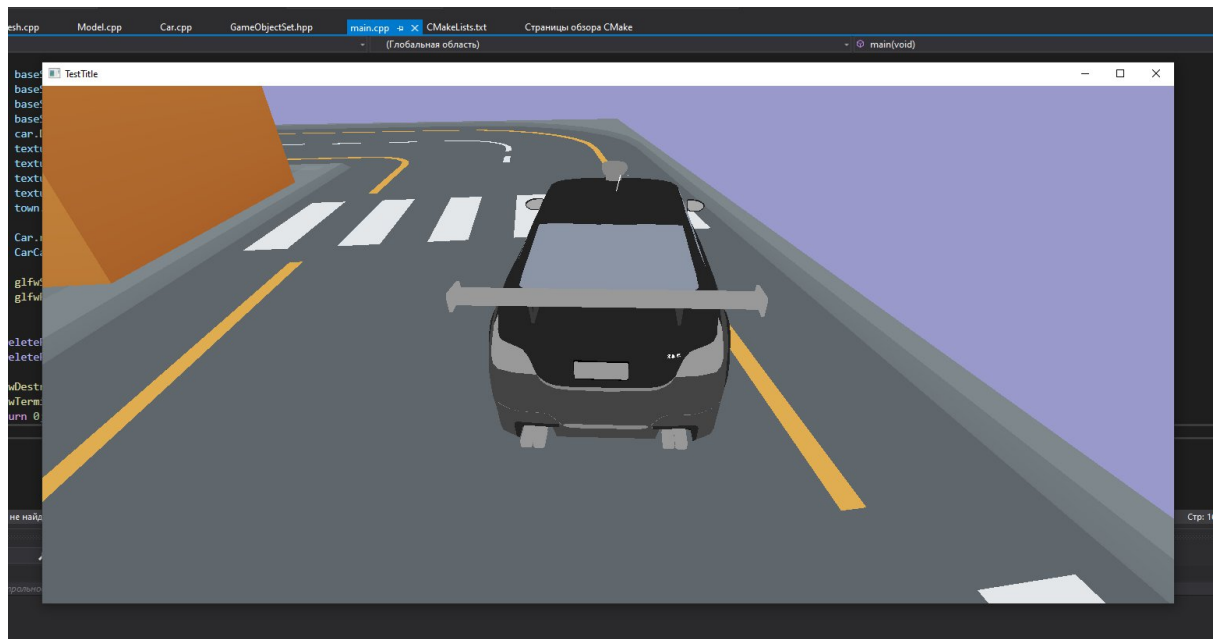


Рисунок 1 - Пример работы программы



Рисунок 2 - Пример работы программы

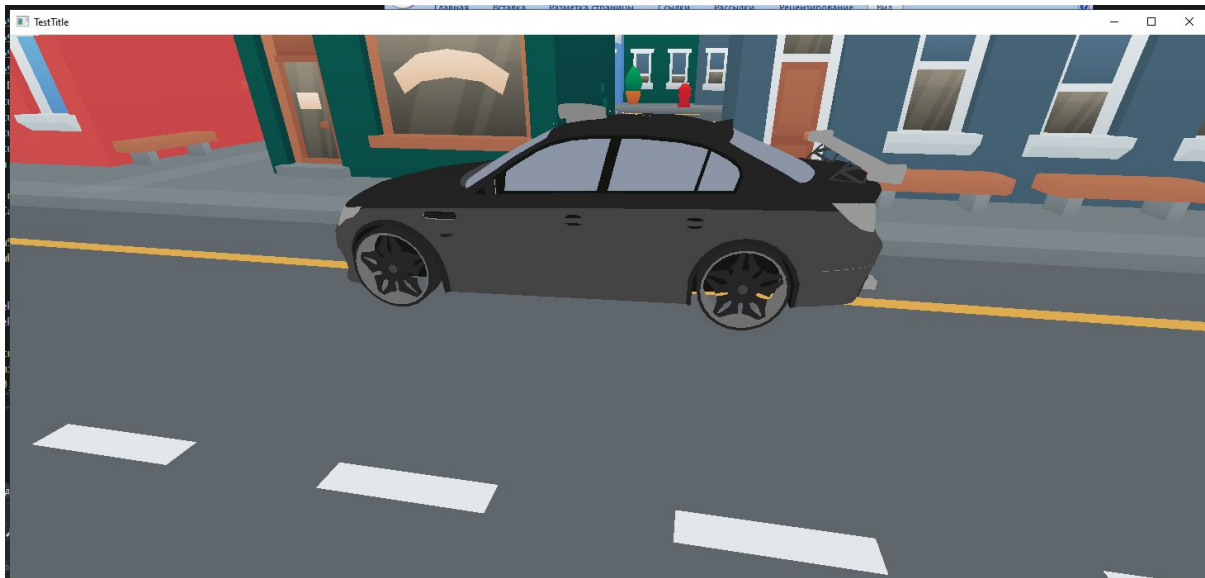


Рисунок 3 - Пример работы программы

Ошибки и неточности

Данная программа содержит невыявленные утечки памяти из буферов GLFW, а так же внутренних буферов экземпляров класса Model. Утечки устранимы, но в дальнейшем в проекте планируется реализация ECS, что сильно повлияет на работу с памятью.

Внутри сцены логика движения автомобиля носит весьма поверхностный характер. Автомобиль не имеет коллизий с окружающей средой, не имеет учёта коэффициента трения поверхностей, не имеет гравитации. «Дорога» является просто моделью, через дома можно проехать, пролететь, за границами карты также возможно движение.

Вывод

В ходе выполнения расчетно-графической работы мной была написана программа, выполняющая импорт моделей и текстур, а так же отрисовку сцены в трёхмерном пространстве с учётом пользовательского ввода. Были получены навыки работы с графической библиотекой GLFW и OpenGL, библиотеками glm, assimp, stb_image и glad.

Список литературы

1.Подбельский В.В., Фомин С.С. Программирование на языке Си: Учеб.пособие.2-е доп.Изд. – М.: Финансы и статистика, 2004 – 600 с.

2.Романов Е. Л. Си/Си++. От дилетанта до профессионала. Электронное учебное "Программирование", "Технология программирования" для студентов 1–2 курсов направления 230100 : учеб. пособие / Е. Л. Романов. – Новосибирский государственный технический университет, № гос регистрации 0321000528, 2010 - 581 с.

3.Си/Си++ от дилетанта до профессионала [Электронный ресурс].
URL: <http://ermak.cs.nstu.ru/cprog/HTML/index.htm>

4. LearnOpenGL [Электронный ресурс].
URL: <https://learnopengl.com/>

Приложение А. Исходный код программы

Mesh.hpp:

```
#ifndef MESH_HPP
#define MESH_HPP

#include <vector>
#include <cstdint>
#include <string>

#include <glm/glm.hpp>

#include "Shader.hpp"

namespace Engine::Render {
    /*
    Создадим enum, чтобы упростить нашу работу на момент импорта текстур.
    */
    enum TextureTypes {
        TEXTURE_DIFFUSE_T = 1 << 0,
        TEXTURE_SPECULAR_T = 1 << 1,
        TEXTURE_NORMAL_T = 1 << 2,
        TEXTURE_HEIGHT_T = 1 << 3,
    };

    /*
    Структура данных точки(вертекса) модели
    */
    struct Vertex {
        glm::vec3 Pos; //позиция.xyz
        glm::vec3 Normal; //нормаль(импортирована).xyz
        glm::vec2 UV; //текстурные координаты .xu (.uv)
        glm::vec4 Color; //цвет вертекса (порой используется в создании
модели).rgba
    };

    /*
    Структура данных текстуры в буфере GLFW
    */
    struct Texture {
        uint32_t id; //имя в адресном пространстве буфера
        TextureTypes type; //тип текстуры (согласно TextureTypes)
        int width, height, channels; //параметры размера импортированного
изображения и количества цветовых каналов.
        std::string path; //путь до текстуры(используется при дедупликации
текстур на этапе импорта)
    };

    class Mesh {
```

```

public:
    std::vector<Vertex> vertices; //вектор точек
    std::vector<uint32_t> indices; //вектор очередности точек при
    построении сетки
    std::vector<Texture> textures; //вектор текстур
    /*
    конструктор класса
    */
    Mesh(const std::vector<Vertex> &vertices, const std::vector<uint32_t>
    &indices, const std::vector<Texture> &textures);
    void Draw(Shader &shader); //функция отрисовки
    //Имя итогового массива вертексов(и их атрибутов) в адресном
    пространстве GLFW, вынесено в public потому что иногда нужно
    uint32_t VAO;

private:
    uint32_t VBO, EBO; //имена массивов позиций точек и их очередности
    void construct(); //вызывается в конструкторе.
};
}

#endif

```

Mesh.cpp:

```
#include <cstdint>
#include <string>

#include <glad/glad.h>

#include "Mesh.hpp"

namespace Engine::Render {
    Mesh::Mesh(const std::vector<Vertex> &vertices, const std::vector<uint32_t>
    &indices, const std::vector<Texture> &textures) {
        this->vertices = vertices;
        this->indices = indices;
        this->textures = textures;
        /*
        собираем из переданных параметров нашу сетку, вызываем construct()
        */
        construct();
    }

    void Mesh::construct() {
        glGenVertexArrays(1, &VAO); //выделить место под VAO в адресах GLFW
        glGenBuffers(1, &VBO); //выделить место под VBO
        glGenBuffers(1, &EBO); //выделить место под EBO

        glBindVertexArray(VAO); //==сейчас работаем с массивом под этим id
        //(помним, что typeof(VAO) == uint32_t)

        glBindBuffer(GL_ARRAY_BUFFER, VBO); //==сейчас работаем с этим
        буфером.
        /*
        как здорово, что в VBO можно сразу залить весь вектор, и он будет
        работать.
        STATIC_DRAW говорит что эти данные не будут меняться после.
        */
        glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(Vertex),
        vertices.data(), GL_STATIC_DRAW);

        glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO); //==сейчас работаем с
        этим буфером.
        /*
        Аналогичная операция для EBO.
        */
        glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.size()*sizeof(uint32_t),
        indices.data(), GL_STATIC_DRAW);

        glEnableVertexAttribArray(0); //включаем, условно, массив атрибутов [0],
        в пространстве VAO.
        /*
```

благодаря тому что вертекс хранит данные в виде различных видов glm:: мы можем пользоваться offsetof(). Новые данные уже не загружаются, мы просто говорим как они расположены в VBO.

```
*/
/*
0 - индекс буфера, 3 - количество элементов, GL_FLOAT - тип элемента
sizeof() - везде один и тот же, мы просто говорим,
что следующий такой элемент встретится через n байт.
offsetof - получаем указатель, который сдвинет нам память на сколько
нужно от начала структуры
*/
```

```
glVertexAttribPointer(0,3, GL_FLOAT, GL_FALSE, sizeof(Vertex), (void*)
offsetof(Vertex, Pos));
```

```
glEnableVertexAttribArray(1);
glVertexAttribPointer(1,3, GL_FLOAT, GL_FALSE, sizeof(Vertex), (void*)
offsetof(Vertex, Normal));
```

```
glEnableVertexAttribArray(2);
glVertexAttribPointer(2,2, GL_FLOAT, GL_FALSE, sizeof(Vertex), (void*)
offsetof(Vertex, UV));
```

```
glEnableVertexAttribArray(3);
glVertexAttribPointer(3,4, GL_FLOAT, GL_FALSE, sizeof(Vertex), (void*)
offsetof(Vertex, Color));
```

//хорошей практикой считается освобождать привязку массивов после работы.

```
glBindVertexArray(0);
}
```

```
void Mesh::Draw(Shader& shader) {
```

```
/*
шейдеры хранят uniform поимённо, поэтому мы прямо в момент
отрисовки конструируем их имена.
всё это основано только на замысле конкретного программиста, лишь бы
работало
```

```
*/
uint32_t diffuseNum = 1;
uint32_t specularNum = 1;
uint32_t normalNum = 1;
uint32_t heightNum = 1;

for (uint32_t i = 0; i < textures.size(); i++) {
    //говорим, что сейчас в шейдер пойдёт текстура за номером i(у
шейдера свой layout)
    glActiveTexture(GL_TEXTURE0 + i);
```

```

        std::string uniformName;
        if (textures[i].type == TextureTypes::TEXTURE_DIFFUSE_T) {
            uniformName =
std::string("texture_diffuse")+std::to_string(diffuseNum++);
        } else if (textures[i].type == TextureTypes::TEXTURE_SPECULAR_T) {
            uniformName =
std::string("texture_specular")+std::to_string(specularNum++);
        } else if (textures[i].type == TextureTypes::TEXTURE_NORMAL_T) {
            uniformName =
std::string("texture_normal")+std::to_string(normalNum++);
        } else if (textures[i].type == TextureTypes::TEXTURE_HEIGHT_T) {
            uniformName =
std::string("texture_height")+std::to_string(heightNum++);
        }
        //прибиваем в память названную текстуру
        glBindTexture(GL_TEXTURE_2D, textures[i].id);
        /*
        говорим, что в шейдере на позиции uniformName расположена наша
        текстура.
        i не слишком важен, мы можем засунуть туда любое значение,
        лишь бы они отличались от uniform к uniform, поскольку
        данное значение теперь прибито гвоздями к нашей текстуре. (как и
        её.id, но в памяти шейдера)
        */
        shader.setInt(uniformName.c_str(), i);
    }
    //Говорим, что сейчас будем работать с массивом в GLFW-памяти, по
    адресу из нашего VAO.
    glBindVertexArray(this->VAO);
    /*
    Рисуем треугольники, точек будет столько, сколько есть в indices
    GL_UNSIGNED_INT - тип данных indices.
    */
    glDrawElements(GL_TRIANGLES, this->indices.size(), GL_UNSIGNED_INT, 0);
    //опять-таки, сбрасываем связь с массивом
    glBindVertexArray(0);
    //и с текстурами!
    glActiveTexture(GL_TEXTURE0);
}
}

```

Model.hpp:

```
#ifndef MODEL_HPP
#define MODEL_HPP

#include <cstdint>

#include <assimp/Importer.hpp>
#include <assimp/scene.h>
#include <assimp/mesh.h>
#include <assimp/material.h>

#include "Mesh.hpp"
#include "Shader.hpp"

namespace Engine::Render {
    class Model {
    public:
        Model(const char* path); //конструктор
        void Draw(Shader &shader); // "рисователь" модели, передаст всё в
        меши.
        std::vector<Mesh> meshes; //вектор из сеток данной модели

    private:
        std::vector<Texture> loadedTextures; //вектор текстур, загруженных при
        импорте
        std::string dir; //строка, используемая чтобы разрешать пути текстур

        void load(const char* path); //вызывается в конструкторе
        void processNode(aiNode* node, const aiScene* scene); //вызывается в
        load, работаем с assimp
        Mesh processMesh(aiMesh* mesh, const aiScene* scene); //вызывается в
        processNode, работаем с assimp
        /*
        вызывается в processMesh, работаем с assimp
        */
        std::vector<Texture> loadMaterialTextures(aiMaterial* mat, aiTextureType
        aiType, TextureTypes engineType);
    };
    /*
    Вызывается в loadMaterialTextures, но оставлена вне модели
    т.к. де-юре не её часть.
    Работаем с assimp и stbi.
    */
    uint32_t LoadTexture(const char* path, const std::string& dir);
}

#endif
```

Model.cpp:

```
#include <cstdint>
#include <string>
#include <cstdio>
#include <vector>

#include <assimp/Importer.hpp>
#include <assimp/postprocess.h>
#include <assimp/scene.h>

#include <assimp/color4.h>
#include <assimp/material.h>
#include <assimp/mesh.h>
#include <assimp/scene.h>
#include <assimp/types.h>

#include <glad/glad.h>

#include <stbi/stb_image.h>

#include "Model.hpp"
#include "Shader.hpp"

namespace Engine::Render {
    Model::Model(const char* path) {
        //да, конструктор вызывает load и больше ничего, можно бы и сократить
        Model::load(path);
    }

    void Model::Draw(Shader &shader) {
        /*
        тоже выглядит понятно, просто для каждой сетки вызываем draw и
        передаём шейдер.
        */
        for (uint32_t i = 0; i < meshes.size(); i++) {
            meshes[i].Draw(shader);
        }
    }

    void Model::load(const char* path) {
        Assimp::Importer importer; //ассимповский класс, дай боже здоровья
        /*
        модификаторы:
        aiProcess_Triangulate - если найдены полигоны более чем из трёх точек -
        сводим к трём.
        aiProcess_FlipUVs - специфическая история для GL-пространства, UV
        OpenGL и DX11 отличаются.
        */
    }
}
```

aiProcess_GenNormals - если при экспорте потеряли, генерируем нормали
aiProcess_GenUVCoords - если при экспорте потеряли, генерируем UV
aiProcess_FindDegenerates - если кто-то забыл точки или сделал кривую сетку - правим
aiProcess_FindInvalidData - если есть проблемы с структурой данных в файле - правим
aiProcess_FixInfacingNormals - если забыли развернуть нормали - правим (работает только на кубах и подобном)

```

*/
const aiScene *scene = importer.ReadFile(path,
aiProcess_Triangulate|aiProcess_FlipUVs|aiProcess_GenNormals|aiProcess_Gen
UVCoords|aiProcess_FindDegenerates|aiProcess_FindInvalidData|aiProcess_FixIn
facingNormals);

if (!scene || scene->mFlags & AI_SCENE_FLAGS_INCOMPLETE
|| !scene->mRootNode) {
    fprintf(stderr, "Error: Assimp: %s\n", importer.GetErrorString());
    //Если сцену не получилось загрузить - ругаемся и выходим.
    return;
}

dir = std::string(path);
//штука для текстур, расскажем позже
//а в целом - просто убираем название модели и говорим, что теперь
работаем вот отсюда
dir = dir.substr(0,dir.find_last_of('/'));

//assimp загружает нам что-то в виде дерева, начинаем с корня
обрабатывать полученное
processNode(scene->mRootNode, scene);
}

void Model::processNode(aiNode* node, const aiScene* scene) {
    for (uint32_t i = 0; i < node->mNumMeshes; i++) {
        //вытащили меш, сунули себе. processMesh будет дальше.
        aiMesh* mesh = scene->mMeshes[node->mMeshes[i]];
        meshes.push_back(processMesh(mesh, scene));
    }

    for (uint32_t i = 0; i < node->mNumChildren; i++) {
        //обрабатываем потомков
        //рекурсия крута
        processNode(node->mChildren[i], scene);
    }
}

Mesh Model::processMesh(aiMesh* mesh, const aiScene* scene) {
    std::vector<Vertex> vertices;
    std::vector<uint32_t> indices;
    std::vector<Texture> textures;

```

```

/*
собираем меш в данных glm:: из данных assimp::
*/
for (uint32_t i = 0; i < mesh->mNumVertices; i++) {
    Vertex vertex;
    vertex.Pos = glm::vec3(
        mesh->mVertices[i].x,
        mesh->mVertices[i].y,
        mesh->mVertices[i].z
    );
    vertex.Normal = glm::vec3(
        mesh->mNormals[i].x,
        mesh->mNormals[i].y,
        mesh->mNormals[i].z
    );
    if (mesh->mTextureCoords[0]) {
        vertex.UV = glm::vec2(
            mesh->mTextureCoords[0][i].x,
            mesh->mTextureCoords[0][i].y
        );
    } else {
        //Если даже тут нет UV - тыкнем в середину, может хоть чего увидим
        vertex.UV = glm::vec2(0.5f, 0.5f);
    }
    vertices.push_back(vertex); //вертекс готов, съём его в вектор
}

//тут тоже всё понятно, разве что static_cast, опять же, приведёт
ассимповский тип к нашему.
for (uint32_t i = 0; i < mesh->mNumFaces; i++) {
    for (uint32_t j = 0; j < mesh->mFaces[i].mNumIndices; j++) {
        indices.push_back(static_cast<uint32_t>(mesh->mFaces[i].mIndices[j]));
    }
}

/*
самое интересное
материал - сложная штука, разный бывает и параметров немеряно
тут - самый тупой вариант
*/
if (mesh->mMaterialIndex >= 0) {
    /*
    вытащили из сцены материал по индексу
    он может повторяться, поэтому в aiNode не лежит.
    */
    aiMaterial* material = scene->mMaterials[mesh->mMaterialIndex];
    /*
    вытащили из материала цвет, если есть

```

```

        засунули его в цвет вертексов, что не совсем верно, но задачу
выполняет
    */
    for (uint32_t i = 0; i < vertices.size(); i++) {
        aiColor4D color;
        aiGetMaterialColor(material, AI_MATKEY_COLOR_DIFFUSE, &color);
        vertices[i].Color.r = color.r;
        vertices[i].Color.g = color.g;
        vertices[i].Color.b = color.b;
    }
    //Загружаем текстуры, подсовывая их пачкой в вектор текстур.
    std::vector<Texture> diffMaps = loadMaterialTextures(material,
aiTextureType_DIFFUSE, TextureTypes::TEXTURE_DIFFUSE_T);
    textures.insert(textures.end(), diffMaps.begin(), diffMaps.end());
    std::vector<Texture> normalMaps = loadMaterialTextures(material,
aiTextureType_NORMALS, TextureTypes::TEXTURE_NORMAL_T);
    textures.insert(textures.end(), normalMaps.begin(), normalMaps.end());
    std::vector<Texture> specularMaps = loadMaterialTextures(material,
aiTextureType_SPECULAR, TextureTypes::TEXTURE_SPECULAR_T);
    textures.insert(textures.end(), specularMaps.begin(), specularMaps.end());
    std::vector<Texture> heightMaps = loadMaterialTextures(material,
aiTextureType_HEIGHT, TextureTypes::TEXTURE_HEIGHT_T);
    textures.insert(textures.end(), heightMaps.begin(), heightMaps.end());
}

    //возвращаем меш, который нам отдаёт конструктор как rvalue. немного
стыдно.
    return Mesh(vertices, indices, textures);
}

```

```

std::vector<Texture> Model::loadMaterialTextures(aiMaterial* mat,
aiTextureType aiType, TextureTypes engineType) {
    std::vector<Texture> textures; //ещё один вектор текстур, потому что надо
    for(uint32_t i = 0; i < mat->GetTextureCount(aiType); i++) {
        aiString str;
        //вытащили название файла текстуры
        mat->GetTexture(aiType, i, &str);
        bool skip = false; //пользуем чтобы пропустить, если уже есть
        //среди загруженных
        for (uint32_t j = 0; j < loadedTextures.size() && !skip; j++) {
            //ищем такие, у которых путь совпадает с текущим
            if (std::strcmp(loadedTextures[j].path.data(), str.C_Str()) == 0) {
                //если нашли - в наш вектор суётся уже загруженная, загрузку
пропускаем
                textures.push_back(loadedTextures[j]);
                skip = true;
            }
        }
    }
}

```

```

        //Если всё же не нашли
        if (!skip) {
            //создали текстуру
            Texture texture;
            //загрузили, получили GLFW-айдишник
            texture.id = LoadTexture(str.C_Str(), dir);
            //тип получили ещё на входе
            texture.type = engineType;
            //путь, ну, понятно
            texture.path = str.C_Str();
            //засунули в текущий вектор и в общий тоже
            textures.push_back(texture);
            loadedTextures.push_back(texture);
        }
    }

    //возвращаем вектор отсюда наверх
    return textures;
}

uint32_t LoadTexture(const char* path, const std::string &dir) {
    std::string fname = std::string(path);
    fname = dir+'/' + fname; //dir нужен именно за этим, нам отдаёт только
название текстуры
    uint32_t textureID; //айди, ничего нового
    glGenTextures(1, &textureID); //говорим, что вот теперь хотим буфер GL-
пространства под текстуру
    int width, height, varAmount; //завели переменные аналогичной логики
что и в нашей структуре
    /*
    загрузили кучу символов, спасибо stbi
    */
    unsigned char* data = stbi_load(fname.c_str(), &width, &height,
    &varAmount, 0);

    if (data) {
        /*
        Тут определили, что у нас с каналами
        нам может прийти одноканальная хайтмапа
        двухканальное что-то, но я не могу придумать что
        ну и обычные текстурки в RGB и RGBA
        */
        GLenum format = varAmount == 1 ? GL_RED :
            varAmount == 3 ? GL_RGB :
            varAmount == 4 ? GL_RGBA:
            GL_RG;
        glBindTexture(GL_TEXTURE_2D, textureID); //==сейчас работаем с
буфером, который сделали ранее
    }
    /*

```

```

        суём в этот буфер 2D!!! текстуру наших параметров
        GL_UNSIGNED_BYTE потому что дата - набор чаров.
        */
        glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0, format,
GL_UNSIGNED_BYTE, data);
        /*
        заставили GLFW придумать мипмапы для этой текстуры
        чтобы меньше места на видеокарте кушало
        в сущности - изображения в меньшем разрешении, в несколько этапов.
        */
        glGenerateMipmap(GL_TEXTURE_2D);

        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
//сказали что повторяемся по одной оси
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
//сказали что повторяемся по другой
        /*
        говорим, что фильтрация пикселей вдали делается линейно,
мипмапами
        вблизи просто линейно
        */
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR_MIPMAP_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    } else {
        //если поломались - поругаемся
        fprintf(stderr, "Error: stbi: failed to load: %s\n", path);
    }
    //удаляем данные stbi, нам они уже не нужны и даром.
    stbi_image_free(data);
    //вернули айдишник, вся текстура уже в памяти
    return textureID;
}
}

```

```

        Shader.hpp:

#ifndef SHADER_HPP
#define SHADER_HPP

#include <cstdint>
#include <string>

#include <glm/glm.hpp>

namespace Engine::Render {
    class Shader {
    public:
        uint32_t ID; //адрес в GLFW-пространстве
        //конструктор
        Shader(const char* vertexShaderPath, const char* fragmentShaderPath);
        //gl_useProgram(id), говорим что работаем сейчас с этим шейдером
        void use();
        //куча функций которые устанавливают uniform нужных типов
        //const подразумевает, что мы не поменяем данные класса
        void setBool(const std::string &name, bool value) const;
        void setInt(const std::string &name, int value) const;
        void setFloat(const std::string &name, float value) const;
        void setVec2(const std::string &name, const glm::vec2 &value) const;
        void setVec3(const std::string &name, const glm::vec3 &value) const;
        void setVec4(const std::string &name, const glm::vec4 &value) const;
        void setMat2(const std::string &name, const glm::mat2 &value) const;
        void setMat3(const std::string &name, const glm::mat3 &value) const;
        void setMat4(const std::string &name, const glm::mat4 &value) const;

    private:
        //маленький еenum чтобы было приятно
        enum ShaderType {
            SHADER_T_VERT = 1 << 0,
            SHADER_T_FRAG = 1 << 1,
        };
        //читаем шейдер из файла
        std::string readShader(const char* path, ShaderType type);
    };
}
#endif

```

Shader.cpp:

```
#include <cstdint>
#include <cstdio>
#include <iostream>
#include <fstream>
#include <sstream>

#include <glad/glad.h>

#include <glm/gtc/type_ptr.hpp>

#include "Shader.hpp"

namespace Engine::Render {
    //чтение из файла
    std::string Shader::readShader(const char* path, ShaderType type) {
        //захотели поработать в плюсовых стрингах, разнообразия ради
        std::string code;
        //установили эээ дескриптор потока, назвали file
        std::ifstream file;
        //сказали, что ошибками считать битость файла
        file.exceptions(std::ifstream::failbit | std::ifstream::badbit);
        try {
            //открыли
            file.open(path);
            //создали поток
            std::stringstream stream;
            //засунули буфер из файла в, по сути, basic_string
            stream << file.rdbuf();
            //закрыли
            file.close();
            //засунули из потока в, вот диво-то, basic_string
            code = stream.str();
        } catch (std::ifstream::failure err) {
            //если стало плохо, ругаемся
            fprintf(stderr, "Error: Shader: %s: Cannot read!\n", path);
            //и чистим что успело записаться, на всякий
            code = std::string("");
        }
        //вернули записанное
        return code;
    }

    //конструктор, компиляция
    Shader::Shader(const char* vertexShaderPath, const char*
    fragmentShaderPath) {
        /*
        получили коды, тип могли бы и не совать
        */
    }
}
```

```

*/
std::string vShaderCode = readShader(vertexShaderPath,
Shader::ShaderType::SHADER_T_VERT);
std::string fShaderCode = readShader(fragmentShaderPath,
Shader::ShaderType::SHADER_T_FRAG);

//собрали массив указателей, так чуть проще работать
const char* vfS_cstring[2] = {vShaderCode.c_str(), fShaderCode.c_str()};
//собрали массив айдишников, аналогично
uint32_t SID[2] = {};
//флаг
int success;
//используем при проблемах
char info[BUFSIZ];

for(uint32_t i = 0; i < 2; i++) {
    //сказали, что наш айдишник - результат генерации места под шейдер
    //нашего типа.
    SID[i] = glCreateShader(i == 0 ? GL_VERTEX_SHADER :
GL_FRAGMENT_SHADER);
    //сказали, что текст брать из строки [i]
    glShaderSource(SID[i], 1, &(vfS_cstring[i]), NULL);
    //скомпилировали
    glCompileShader(SID[i]);
    //получаем инфу об ошибках компиляции
    glGetShaderiv(SID[i], GL_COMPILE_STATUS, &success);
    if (!success) {
        //если сломалось - пишем в info, что сломалось
        glGetShaderInfoLog(SID[i], BUFSIZ, NULL, info);
        //небольшой переключатель между двумя загруженными путями
        const char* patherr = i == 0 ? vertexShaderPath : fragmentShaderPath;
        //пугаемся
        fprintf(stderr, "Error: Shader: %s: Cannot compile!:\n%s\n", patherr,
info);
        //удаляем лишнее сразу
        glDeleteShader(SID[i]);
    }
}

//айдишник основного шейдера привязываем тут
ID = glCreateProgram();
//к "программе" привязываем два наших шейдера
glAttachShader(ID, SID[0]);
glAttachShader(ID, SID[1]);
//и привязываем программу к потоку отрисовки
glLinkProgram(ID);
//Спросили, всё ли ок
glGetProgramiv(ID, GL_LINK_STATUS, &success);
if (!success) {

```

```

        //если не ок - получаем инфу и ругаемся
        glGetProgramInfoLog(ID, BUFSIZ, NULL, info);
        fprintf(stderr, "Error: Shader: PID: %u: Cannot link!:\n%s\n", ID, info);
    }
    /*
    если ранее был успех-победа, то шейдеры можно удалять, они уже в
программе
    */
    if (success) {
        glDeleteShader(SID[0]);
        glDeleteShader(SID[1]);
    }
}

//glUseProgram(id), говорим что работаем сейчас с этим шейдером
void Shader::use() {
    glUseProgram(ID);
}

/*
Следующие функции однотипны, придерживаются единой логики:
установить_значение(по_адресу(шейдер, название_переменной),
значение)
*/
void Shader::setBool(const std::string &name, bool value) const
{
    glUniform1i(glGetUniformLocation(ID, name.c_str()), (int)value);
}
void Shader::setInt(const std::string &name, int value) const
{
    glUniform1i(glGetUniformLocation(ID, name.c_str()), value);
}
void Shader::setFloat(const std::string &name, float value) const
{
    glUniform1f(glGetUniformLocation(ID, name.c_str()), value);
}
void Shader::setVec2(const std::string &name, const glm::vec2 &value) const
{
    glUniform2fv(glGetUniformLocation(ID, name.c_str()), 1, &value[0]);
}
void Shader::setVec3(const std::string &name, const glm::vec3 &value) const
{
    glUniform3fv(glGetUniformLocation(ID, name.c_str()), 1, &value[0]);
}
void Shader::setVec4(const std::string &name, const glm::vec4 &value) const
{
    glUniform4fv(glGetUniformLocation(ID, name.c_str()), 1, &value[0]);
}

```

```

void Shader::setMat2(const std::string &name, const glm::mat2 &mat) const
{
    glUniformMatrix2fv(glGetUniformLocation(ID, name.c_str()), 1, GL_FALSE,
glm::value_ptr(mat));
}
void Shader::setMat3(const std::string &name, const glm::mat3 &mat) const
{
    glUniformMatrix3fv(glGetUniformLocation(ID, name.c_str()), 1, GL_FALSE,
glm::value_ptr(mat));
}
void Shader::setMat4(const std::string &name, const glm::mat4 &mat) const
{
    glUniformMatrix4fv(glGetUniformLocation(ID, name.c_str()), 1, GL_FALSE,
glm::value_ptr(mat));
}
}

```

Components.hpp:

```
#ifndef COMPONENTS_HPP
#define COMPONENTS_HPP

#include <glm/glm.hpp>

namespace JustDrive::Components {
    class Transform {
    public:
        Transform(); //конструктор, отдающий нули
        Transform(glm::vec3 position); //конструктор, пишущий позицию сразу

        /*
        своего рода геттеры-сеттеры нужных переменных
        */
        glm::vec3 getPosition();
        void setPosition(glm::vec3 position);
        glm::vec3 getRotation();
        void setRotation(glm::vec3 radians);
        glm::vec3 getScale();
        void setScale(glm::vec3 axis_scale);

        /*
        геттеры векторов ориентации, сеттеры не предполагаются
        */
        glm::vec3 getForward();
        glm::vec3 getUp();
        glm::vec3 getRight();
        //Тут конструируем матрицу Model
        glm::mat4 getTransform();

    private:
        glm::vec3 position;//позиция
        glm::vec3 rotation;//поворот
        glm::vec3 scale;//масштаб

        glm::vec3 forward;//вектор "вперёд"
        glm::vec3 up;//вектор "вправо" (считаем что "вверх" положителен)
        glm::vec3 right;//вектор "вверх"

        void recountVectors(); //это пересчитает векторы ориентации,
        вызываем из сеттеров.

    };
}

#endif
```

Components.cpp:

```
#include "Components.hpp"
#include "glm/ext/matrix_transform.hpp"
#include <glm/gtc/matrix_transform.hpp>
#include <glm/geometric.hpp>
#include <glm/gtc/quaternion.hpp>
#include <glm/trigonometric.hpp>
#define GLM_ENABLE_EXPERIMENTAL
#include <glm/gtx/matrix_decompose.hpp>

namespace JustDrive::Components {

    /*
    ВЫГЛЯДИТ ЛОГИЧНО
    */
    Transform::Transform() {
        position = glm::vec3(0.0f);
        rotation = glm::vec3(0.0f);
        scale = glm::vec3(1.0f, 1.0f, 1.0f);
        recountVectors();
    }

    /*
    ВЫГЛЯДИТ ЛОГИЧНО
    */
    Transform::Transform(glm::vec3 Position) {
        position = Position;
        rotation = glm::vec3(0.0f);
        scale = glm::vec3(1.0f, 1.0f, 1.0f);
        recountVectors();
    }

    /*
    ВЫГЛЯДИТ ЛОГИЧНО
    */
    glm::vec3 Transform::getPosition() {
        return position;
    }

    /*
    ВЫГЛЯДИТ ЛОГИЧНО
    */
    glm::vec3 Transform::getRotation() {
        return rotation;
    }

    /*
    ВЫГЛЯДИТ ЛОГИЧНО
    */
```

```

*/
glm::vec3 Transform::getScale() {
    return scale;
}
/*
выглядит логично
*/
void Transform::setPosition(glm::vec3 new_position) {
    position = new_position;
}
/*
тут пересчитываем, чтобы держать нашу ориентацию правильной
*/
void Transform::setRotation(glm::vec3 angles_radian) {
    rotation = angles_radian;
    recountVectors();
}
/*
тут пересчитываем на всякий случай, но вообще - не надо
если честно, это осталось с начала проекта двухмесячной давности
*/
void Transform::setScale(glm::vec3 axis_scale) {
    scale = axis_scale;
    recountVectors();
}
/*
выглядит логично
*/
glm::vec3 Transform::getForward() {
    return forward;
}
/*
выглядит логично
*/
glm::vec3 Transform::getUp() {
    return up;
}
/*
выглядит логично
*/
glm::vec3 Transform::getRight() {
    return right;
}
/*
как делаем матрицу:
берём единичную, её масштабируем
потом перемещаем, ведь нас интересует поворачиваться на месте, а не от
центра
потом _поворачиваем_, по трём осям в отдельности

```

наверное, можно и вместе как-то, кватернионами к примеру
но так оно понятнее выглядит

```
*/
glm::mat4 Transform::getTransform() {
    glm::mat4 transform = glm::scale(glm::mat4(1.0f), scale);
    transform = glm::translate(transform, position);
    transform = glm::rotate(transform, rotation.y, glm::vec3(0.0f,1.0f, 0.0f));
    transform = glm::rotate(transform, rotation.x, glm::vec3(1.0f,0.0f, 0.0f));
    transform = glm::rotate(transform, rotation.z, glm::vec3(0.0f,0.0f, 1.0f));
    return transform;
}

void Transform::recountVectors() {
    /*
    наши любимые матрицы поворотов вдоль оси
    */
    const glm::mat3 rotX(
        1.0f, 0.0f, 0.0f,
        0.0f, glm::cos(rotation.x), -glm::sin(rotation.x),
        0.0f, glm::sin(rotation.x), glm::cos(rotation.x)
    );
    const glm::mat3 rotY(
        glm::cos(rotation.y), 0.0f, -glm::sin(rotation.y),
        0.0f, 1.0f, 0.0f,
        glm::sin(rotation.y), 0.0f, glm::cos(rotation.y)
    );
    const glm::mat3 rotZ(
        glm::cos(rotation.z), -glm::sin(rotation.z), 0.0f,
        glm::sin(rotation.z), glm::cos(rotation.z), 0.0f,
        0.0f, 0.0f, 1.0f
    );
    /*
    ну и вектор вперёд - вектор вида 0,0,z, который
    мы крутим по нашим радианам поворота
    */
    forward = glm::vec3(0.0f,0.0f,1.0f)*rotY*rotX*rotZ;
    //русский не знаю, находим вектор, ортогональный "вперёд" и мировому
    "вверх"
    right = glm::cross(forward, glm::vec3(0.0f, 1.0f, 0.0f));
    /*
    получили вектор "вправо"
    ну а ортогональный ему и "вперёд", что логично - локальный "вверх"
    */
    up = glm::cross(right, forward);
}
}
```

GameObject.hpp:

```
#ifndef GAMEOBJECT_HPP
#define GAMEOBJECT_HPP
#define GLFW_INCLUDE_NONE
#include <GLFW/glfw3.h>

#include <glm/glm.hpp>

#include "Components.hpp"

namespace JustDrive {
    class GameObject {
    public:
        GameObject(); //конструктор
        GameObject(glm::vec3 position); //конструктор с координатами
        GameObject(Components::Transform transform); //конструктор с целой
        трансформой
        //своя трансформа
        Components::Transform transform;
        //функции для потомков, поэтому тела нет.
        virtual void initialize(GLFWwindow* window) {};
        virtual void run(GLFWwindow* window, float deltaTime) {};
    };
}

#endif
```

GameObject.cpp:

```
#include "GameObject.hpp"
#include "Components.hpp"

namespace JustDrive {
    GameObject::GameObject() {
        transform = Components::Transform();
    }

    GameObject::GameObject(Components::Transform Transform) {
        transform = Transform;
    }

    GameObject::GameObject(glm::vec3 Position) {
        transform = Components::Transform(Position);
    }
}
```

Car.hpp:

```
#ifndef CAR_HPP
#define CAR_HPP

#include "GameObject.hpp"

namespace JustDrive {
    class Car : public GameObject {
    public:
        Car( //конструктор
            glm::vec3 position,
            glm::vec3 rotation_rad,
            float maxSpeedForward,
            float maxSpeedbackward,
            float acceleration,
            float tractionParallel,
            float tractionCross,
            float maxWheelAngle
        );
        //перезаписываю полученное от GameObject
        virtual void initialize(GLFWwindow* window) override;
        virtual void run(GLFWwindow* window, float deltaTime) override;
        /*
        тут вернули свою трансформу, потому что машинка - кривая
        с точки зрения ООП это делать неправильно, и вообще должно
        управляться не отсюда
        */
        glm::mat4 getTransform();
    private:
        float maxSpeedForward; //макс. скорость вперёд
        float maxSpeedBackward; //макс. скорость назад
        float acceleration; //ускорение
        float tractionParallel; //трение вдоль "переда"
        float tractionCross; //трение ортогонально "переду"
        float maxWheelAngle; //угол поворота колёс, максимальный
        glm::vec3 currentVelocity; //текущая скорость, локальная
        float currentWheelAngle; //текущий угол поворота колёс
    };
}

#endif
```

Car.cpp:

```
#include "glm/common.hpp"
#include "glm/ext/matrix_transform.hpp"
#include "glm/geometric.hpp"
#define GLFW_INCLUDE_NONE
#include <GLFW/glfw3.h>

#include <glm/glm.hpp>
#include <glm/trigonometric.hpp>
#include <glm/gtc/matrix_transform.hpp>

#include "Car.hpp"
#include "Components.hpp"
#include "GameObject.hpp"

#include <cstdio>

namespace JustDrive {
    Car::Car(
        glm::vec3 position,
        glm::vec3 rotation_rad,
        float maxSpeedForward,
        float maxSpeedBackward,
        float acceleration,
        float tractionParallel,
        float tractionCross,
        float mWheelAngle
    ):
        maxSpeedForward(maxSpeedForward),
        maxSpeedBackward(maxSpeedBackward),
        acceleration(acceleration),
        tractionParallel(tractionParallel),
        tractionCross(tractionCross)
    {
        /*
        то что выше мы уже видели, тут - инициализация трансформы
        */
        GameObject::transform = Components::Transform(position);
        GameObject::transform.setRotation(rotation_rad);
        //угол приводим к радианам, да и по модулю возьмём, а то вдруг
        приколы пишут
        maxWheelAngle = glm::radians(glm::abs(mWheelAngle));
        //текущая скорость - 0
        currentVelocity = glm::vec3(0.0f, 0.0f, 0.0f);
        //текущий угол - 0
        currentWheelAngle = 0.0f;
    }
}
```

```

void Car::initialize(GLFWwindow* window) {
    //нам это не надо, но понадобилось бы, если бы мы ждали
инициализации других классов
    return;
}

void Car::run(GLFWwindow* window, float deltaTime) {
    /*
    вектор ввода пользователя:
    .x = w-s (вперёд - назад)
    .y = a-d (влево - вправо)
    */
    glm::vec2 userInput(
        (float)(glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
        -(float)(glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS),
        (float)(glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)
        -(float)(glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)
    );

    float accelerationZ = 0.0f; //ускорение "в кадр"
    //тут также условия применения (вперёд меньше максимальной вперёд,
аналогично назад)
    if ((currentVelocity.z < maxSpeedForward && userInput.x >
0.0f) || (currentVelocity.z > -maxSpeedBackward && userInput.x < 0.0f)) {
        accelerationZ = userInput.x*deltaTime*acceleration;
    } else {
        accelerationZ = 0.0f;
    }

    //ускоряемся (или замедляемся, если в другую сторону жмём)
    currentVelocity.z += accelerationZ;
    //скорость в мировых координатах (по вектору "вперёд")
    glm::vec3 velocity = currentVelocity.z*transform.getForward();
    //у неё .x в другую сторону, и я не знаю почему
    velocity.x = -velocity.x;

    //сменили позицию на +нашу скорость
    transform.setPosition(transform.getPosition()+deltaTime*velocity);

    //считаем угол
    currentWheelAngle = maxWheelAngle*userInput.y;
    currentWheelAngle *= (maxSpeedForward-glm::abs(currentVelocity.z));
    if (glm::abs(currentWheelAngle) < maxWheelAngle/3.0f) {
        currentWheelAngle = userInput.y*maxWheelAngle/3.0f;
    }
    //поворачиваемся вдоль оси Y
    if (currentVelocity.z >= maxSpeedForward/10.0f) //если она меньше, нет
смысла считать, мы там никуда не уедем
    transform.setRotation(transform.getRotation()+glm::vec3(
        0,

```

```

        //Суть такова - чем медленнее, тем острее поворот, поэтому смотрим
на дельту скорости и максимума вперёд
        //А домножаем на смешные числа потому что это _радианы_
        0.3f*currentWheelAngle*deltaTime,
        0
    ));
}

glm::mat4 Car::getTransform() {
    //Вот то, о чём я говорил - машинку приходится масштабировать, потом
крутить, а после ещё и двигать вверх.
    return
glm::translate(glm::rotate(glm::scale(transform.getTransform()),glm::vec3(0.2f,0.2f
,0.2f)), 3.1415f/2.0f, transform.getUp()), glm::vec3(0.0f, 0.4f, 0.0f));
}
}

```

Camera.hpp:

```
#include "Components.hpp"
#include "GameObject.hpp"

namespace JustDrive {
    class CarCamera : public GameObject {
    public:
        //конструктор
        CarCamera(glm::vec3 position, float lerpMultiplier, float minDistance,
Components::Transform* targetPosition);
        //переписываем вещи из предка
        virtual void initialize(GLFWwindow* window) override;
        virtual void run(GLFWwindow* window, float deltaTime) override;
        //тут вернём нашу матрицу для шейдеров
        glm::mat4 getViewMatrix();
        //это пользуем в матрице проекции
        float yFov;
    private:
        //множитель интенсивности интерполяции
        float lerpMultiplier;
        //дистанция до цели, минимальная
        float minDistance;
        //указатель на трансформу цели (как удобно, знаем об изменениях!)
        Components::Transform* targetPosition;
    };
}
```

Camera.cpp:

```
#include <glm/ext/quaternion_common.hpp>
#include <glm/gtc/matrix_transform.hpp>

#include "Camera.hpp"
#include "Components.hpp"
#include "GameObject.hpp"

namespace JustDrive {
    /*
        выглядит логично, поле обзора пока константой тыкнем
    */
    CarCamera::CarCamera(glm::vec3 position, float lerpMultiplier, float
minDistance, Components::Transform* targetPosition) :
lerpMultiplier(lerpMultiplier), minDistance(minDistance),
targetPosition(targetPosition) {
        GameObject::transform = Components::Transform(position);
        yFov = 45.0f;
    }

    void CarCamera::initialize(GLFWwindow* window) {
        //нам не особо надо, а в целом - пригодится
        return;
    }

    void CarCamera::run(GLFWwindow* window, float deltaTime) {
        //наша позиция
        glm::vec3 camPos = GameObject::transform.getPosition();
        /*
            вектор в направлении цели
            +в конце это приколы,
            просто мы хотим смотреть не куда-то в трансмиссию, а желательно в
            область крыши машины.
        */
        glm::vec3 directionVector = targetPosition->getPosition()-
camPos+glm::vec3(0.0f, 0.5f, 0.0f);
        //длина этого вектора - минимальная дистанция, сколько нужно покрыть
        float distanceToCover = glm::length(directionVector)-minDistance;
        //вектор перемещения - покрытие на нормализованный вектор
        направления, что удобно
        glm::vec3 movementVector =
distanceToCover*glm::normalize(directionVector);
        //итоговая позиция цели
        glm::vec3 movementTargetPosition = camPos+movementVector;
        //позиция по итогам кадра, линейная интерполяция текущих и целевых
        координат по deltaTime*lerpMultiplier
    }
```

```

        glm::vec3 frameMovement = camPos +
(deltaTime*lerpMultiplier)*(movementTargetPosition-camPos); // actually /(1-0)
        //меняем только позицию, взгляд за нас getViewMatrix() вернёт.
        GameObject::transform.setPosition(frameMovement);
    }

    glm::mat4 CarCamera::getViewMatrix() {
        //Что я и говорил. В целом lookAt сомнительно справляется со своими
задачами, но нам - подходит.
        return glm::lookAt(GameObject::transform.getPosition(),
targetPosition->getPosition(), GameObject::transform.getUp());
    }
}

```

main.cpp:

```
#define GLFW_INCLUDE_NONE
#include <GLFW/glfw3.h>
#include <glad/glad.h>

#include <stdio.h>

#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>

#include <stbi/stb_image.h>

#include "../Engine/Engine.hpp"
#include "GameObjectSet.hpp"

namespace GLFW {
    namespace Callbacks {
        void Error(int error, const char* desc) {
            //коллбек, расскажет где ошиблись
            fprintf(stderr, "GLFW Error: %s\n", desc);
        }

        //коллбек, базовые элементы управления ходом работы программы
        void Key(GLFWwindow* window, int key, int scancode, int
action, int mods) {
            if (key == GLFW_KEY_0 && action == GLFW_PRESS) {
                glfwSetWindowShouldClose(window, GLFW_TRUE);
            }
            else if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS) {
                glfwSetInputMode(window, GLFW_CURSOR,
GLFW_CURSOR_NORMAL);
            }
            else if (key == GLFW_KEY_LEFT_SHIFT && action ==
GLFW_PRESS) {
                glfwSetInputMode(window, GLFW_CURSOR,
GLFW_CURSOR_HIDDEN);
            }
        }

        //коллбек, позволяет безболезненно менять размер окна
        void FramebufferSizeChanged(GLFWwindow* window, int width, int
height) {
            glViewport(0,0,width,height);
        }
    }

    GLFWwindow* bootstrap(const char* title) {
        glfwSetErrorCallback(Callbacks::Error); //привязали коллбек
        if (!glfwInit()) return 0; //если не получилось создать
контекст - умираем

        //указываем версию, нужно для шейдеров да и в целом хорошая
практика
        glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
        glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
        //указываем, что работать хотим в CORE, а не COMPAT, получаем
функционал
    }
```

```

glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

//создаём окошко базовых размеров
GLFWwindow* window = glfwCreateWindow(640, 480, title, NULL,
NULL);

//если не создалось - убиваемся и умираем
if (!window) {
    glfwTerminate();
    return 0;
}

//говорим, что работаем с этим окошком
glfwMakeContextCurrent(window);

//привязали коллбеки
glfwSetFramebufferSizeCallback(window,
Callbacks::FramebufferSizeChanged);
glfwSetKeyCallback(window, Callbacks::Key);
//говорим, что курсор нужно скушать и не отдавать (по коллбеку
- отдадут)
glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_DISABLED);

//загружаем GLAD
if (!gladLoadGL()) {
    //если не вышло - умираем и убиваемся, дважды.
    fprintf(stderr, "Error: Bootstrap: Cannot load GLAD!\n");
    glfwDestroyWindow(window);
    glfwTerminate();
    return 0;
}
//полезная штучка для импорта моделей, а всё потому что
спасибо OpenGL
stbi_set_flip_vertically_on_load(false);

//Включаем базовое тестирование глубины, чтобы модели поверх
друг друга правильно рисовались
glEnable(GL_DEPTH_TEST);
glDepthFunc(GL_LESS);
//Могли и куллинг включить, но у машинки кривые непоправимые
нормали
/*
glEnable(GL_CULL_FACE);
glCullFace(GL_BACK);
*/
//Говорим, что вертикально хотим синхронизироваться с
герцовой монитор
glfwSwapInterval(1);
//Возвращаем готовое окно
return window;
}

}

int main(void) {
    GLFWwindow* window = GLFW::bootstrap("TestTitle");
    //если окна не получилось - умираем
    if (!window) {

```

```

        return -1;
    }
    /*
    сразу меняем размер. на Windows не страшно,
    а вот Arch+Wayland не дают создать произвольное окно,
    управляя размером сами
    */
    int width, height;
    glfwGetFramebufferSize(window, &width, &height);
    glfwViewport(0, 0, width, height);

    //Инициализировали шейдеры
    Engine::Render::Shader baseShader =
    Engine::Render::Shader("resources/Base.vs", "resources/Base.fs");
    Engine::Render::Shader texturedShader =
    Engine::Render::Shader("resources/Textured.vs",
    "resources/Textured.fs");

    //Инициализировали модели
    Engine::Render::Model car =
    Engine::Render::Model("resources/Models/CarReexported/Car.obj");
    Engine::Render::Model town =
    Engine::Render::Model("resources/Models/Terrain/SmallCity.obj");

    //переменные для кадровой логики
    float deltaTime = 0.0f;
    float lastFrameTime = 0.0f;

    //отчитались что всё классно
    printf("Going to window\n");

    //поставили цвет фона(далее в окошке будет именно он)
    glClearColor(0.6f, 0.6f, 0.8f, 1.0f);

    //Инициализируем автомобиль
    JustDrive::Car Car(
        glm::vec3(0.0f, 0.0f, 0.0f),
        glm::vec3(0.0f, 0.0f, 0.0f),
        5.0f,
        2.0f,
        2.0f,
        0.8f,
        1.3f,
        20.0f
    );
    //Инициализируем камеру(потому что трансформу-то из автомобиля
    надо взять!)
    JustDrive::CarCamera CarCamera(
        glm::vec3(0.0f, -10.0f, 0.0f),
        3.0f,
        1.0f,
        &Car.GameObject::transform
    );

    //Основной цикл отрисовки
    //Условие может выполняться либо по коллбеку, либо по сигналу типа
    -USR1

```

```

while (!glfwWindowShouldClose(window)) {
    //Чистим буферы цвета и глубины
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    float frameTime = static_cast<float>(glfwGetTime());
    deltaTime = frameTime-lastFrameTime;
    lastFrameTime = frameTime;

    //придумали наши матрицы машины, окружения, камеры и проекции
    glm::mat4 townModel, carModel, view, projection;
    townModel = glm::translate(glm::mat4(1.0f), glm::vec3(0.0f,
0.0f, 0.0f));
    townModel = glm::scale(townModel, glm::vec3(1.0f, 1.0f,
1.0f));

    //ну тут понятно, уже говорили
    carModel = Car.getTransform();
    //тут тоже
    view = CarCamera.getViewMatrix();
    //получили размер окошка, для матрицы проекции
    glfwGetFramebufferSize(window, &width, &height);
    //создали матрицу проекции
    projection = glm::perspective(CarCamera.yFov,
(float)width/height, 0.1f, 100.0f);
    //это шейдер автомобиля, делаем:
    baseShader.use();
    //присваиваем матрицы в нужные места в шейдере
    baseShader.setMat4("mProjection", projection);
    baseShader.setMat4("mView", view);
    baseShader.setMat4("mModel", carModel);
    //рисует машину с заданным шейдером
    car.Draw(baseShader);
    //теперь шейдер города, он использует текстуры
    texturedShader.use();
    //аналогично, присваиваем матрицы, разве что mModel другая
    texturedShader.setMat4("mProjection", projection);
    texturedShader.setMat4("mView", view);
    texturedShader.setMat4("mModel", townModel);
    //рисует город с заданным шейдером
    town.Draw(texturedShader);

    //вызываем методы кадрового изменения в нашей машине и
камере
    Car.run(window, deltaTime);
    CarCamera.run(window, deltaTime);

    //подменяем текущий кадр новым
    glfwSwapBuffers(window);
    //получаем "ивенты" - нажатие кнопок, сигналы извне и т.д.
    glfwPollEvents();
}

//чистим память от шейдеров
glDeleteProgram(baseShader.ID);
glDeleteProgram(texturedShader.ID);

```

```
    //а вот буферы GLFW почистит сам в момент glfwTerminate()  
    glfwDestroyWindow(window);  
    glfwTerminate();  
    return 0;  
}
```